Materia: Programación 4

TRABAJO PRÁCTICO INTEGRADOR · FOOD STORE, SISTEMA DE PEDIDOS DE COMIDA

Food Store

Coordinador: Prof. Carlos Martínez

Profesores: Cinthia Rigoni, Neyen Bianchi y Ramiro Hualpa

Objetivos

Construir una aplicación web completa para un negocio de comidas, con un backend FastAPI + SQLModel y un frontend React + TypeScript, aplicando la arquitectura en capas con Unit of Work, la autenticación con JWT en cookie HttpOnly, la autorización por roles, el manejo del estado del servidor con TanStack Query y del estado del cliente con Zustand.

Al finalizar el trabajo se espera que el alumno sea capaz de:

- Modelar un dominio con SQLModel (relaciones 1:N, baja lógica y copia de datos en las líneas de un pedido) a partir de un diagrama UML.
- Implementar reglas de negocio transaccionales (descuento y restitución de stock, máquina de estados) dentro de un Unit of Work.
- Exponer una API REST documentada, con códigos de respuesta correctos y errores sin ambigüedad.
- Consumir esa API desde React con Axios, TanStack Query, TanStack Form y TanStack Table, y proteger las pantallas según el rol.

Descripción del sistema

Food Store es una aplicación web para un negocio de comidas. Los clientes se registran, recorren el catálogo, arman un carrito y confirman un pedido indicando la dirección de entrega y la forma de pago (efectivo o transferencia). El negocio administra categorías, productos, stock, pedidos y usuarios desde un panel de administración.

Actor	Objetivo
Cliente (rol `user`)	Navegar el catálogo, gestionar el carrito, confirmar el pedido con dirección y forma de pago, y consultar y cancelar sus pedidos.
Administrador (rol `admin`)	Gestionar categorías, productos, stock y usuarios, avanzar los pedidos según la máquina de estados y ver un resumen del negocio.

Actor	Objetivo
Sistema	Garantizar que un pedido se cree completo o no se cree, que el stock se descuente y se restituya de forma coherente y que los precios de un pedido no cambien después de creado.

Alcance. Autenticación con JWT en cookie HttpOnly y dos roles; catálogo con una categoría por producto e imagen como URL; carrito persistido en el navegador; pedidos con máquina de 4 estados, descuento de stock en la transacción y restitución al cancelar; pago manual sin pasarela; panel de administración con CRUD de categorías y productos, stock, pedidos, usuarios y resumen; CORS con credenciales, `.env.example` y seed; API documentada en `/docs` y `/redoc`.

Fuera de alcance. Direcciones guardadas por usuario, varias categorías por producto, historial de cambios de estado, gráficos, pasarela de pago, pruebas automatizadas y notificaciones.

Arquitectura general. Dos proyectos independientes que se comunican por HTTP. El backend expone la API REST bajo `/api/v1`, aplica las reglas de negocio y persiste en PostgreSQL. El frontend es una aplicación React que consume la API con Axios. La sesión viaja en una cookie HttpOnly que el navegador adjunta en cada petición: el frontend nunca lee ni guarda el token. El modelo de datos se entrega en el diagrama UML adjunto, `UML_FOOD_STORE` (archivos PNG y SVG), reproducido en el anexo (Figura 1): detalla entidades, campos, tipos, restricciones, relaciones con su multiplicidad y las reglas de negocio en notas.

Se puede partir del proyecto de seguridad con JWT y cookie HttpOnly de la materia: aporta el módulo de usuarios, el login, las dependencias de sesión y rol, CORS con credenciales y el seed.

Flujo de trabajo propuesto

El trabajo se organiza en diez partes, en el orden sugerido para construirlas. Cada parte indica qué se construye; el detalle técnico está en las secciones de especificación que siguen y el puntaje de cada una, en «Criterios de evaluación».

Backend: estructura y configuración

Crear el proyecto `backend/` con la estructura de carpetas de la especificación del backend: un paquete `core/` con la infraestructura compartida y un paquete por dominio en `modules/`. Los módulos `usuarios`, `categorias`, `productos` y `pedidos` tienen router, service, unit of work, repository, models y schemas; `auth` solo router, service y schemas, porque usa el modelo Usuario. Configurar las variables con pydantic-settings leídas de `.env`, el middleware CORS con credenciales, el prefijo `/api/v1` y el seed idempotente. Fijar las versiones de todas las dependencias en `requirements.txt`, incluidas `passlib[bcrypt]==1.7.4` y `bcrypt==4.0.1`.

Modelo de datos

Implementar las cinco entidades del diagrama UML como modelos SQLModel con `table=True`: campos, tipos, valores por defecto, claves y relaciones con los nombres exactos indicados en «Modelos y relaciones». Baja lógica con un campo booleano; los pedidos no se borran. `DetallePedido` guarda una copia del nombre y del precio del producto.

Unit of Work y Repository

Implementar `BaseRepository[T]` y un `UnitOfWork` base que abre la sesión en `__enter__` y hace commit, rollback y close en `__exit__`. En los módulos `categorias`, `productos` y `pedidos`, el Service abre `with self.uow as uow:`, aplica las reglas y devuelve el schema de salida dentro del bloque; el Router obtiene el Service con `Depends` y no abre el Unit of Work. Ningún Service ni Router llama a `session.commit()`.

Autenticación y usuarios

Implementar el registro público, el login por formulario OAuth2 que entrega el JWT en la cookie HttpOnly `access_token`, el logout que borra la cookie y el perfil `/me`, con las dependencias `get_current_user`, `get_current_active_user` y `require_role`. Agregar el listado, la activación y la desactivación de usuarios para el rol `admin`. Las contraseñas se guardan con hash bcrypt. Los códigos de error son los de la especificación: 409 en el registro duplicado, 401 en credenciales incorrectas, 400 en cuenta desactivada y 404 en usuario inexistente.

API de catálogo

Implementar los endpoints de categorías y productos con los verbos, códigos y schemas de la especificación: lecturas públicas que devuelven solo registros activos, escrituras solo para `admin`, listado de productos paginado con búsqueda y filtros, cambio de stock y disponibilidad, y baja lógica con `DELETE` que responde 204.

API de pedidos y reglas de negocio

Implementar la creación transaccional del pedido desde el carrito (RN-01), la máquina de estados (RN-02), la cancelación con restitución de stock (RN-03), los permisos por dueño y rol (RN-04), la inmutabilidad de líneas y total (RN-05), la validación de forma de pago y dirección (RN-06) y el resumen para el panel. Los códigos de error de cada endpoint son los de la especificación.

Frontend: cliente

Construir las pantallas de login, registro, catálogo (paginado en el servidor, con búsqueda, filtro por categoría, skeleton y estado vacío), carrito persistido, checkout con TanStack Form y «Mis pedidos» con detalle y cancelación en PENDIENTE.

Frontend: panel de administración

Construir las pantallas de resumen (tarjetas con los indicadores), ABM de categorías y productos con TanStack Table y TanStack Form (incluido el cambio de stock y disponibilidad), gestión de pedidos con cambio de estado y gestión de usuarios con activar y desactivar.

Frontend: estado, datos y sesión

Acceder a la API con una instancia de Axios con `withCredentials` e interceptor de 401; obtener y modificar los datos del servidor con TanStack Query (`useQuery` , `useMutation` , invalidación); mantener la sesión en `authStore` y el carrito en `cartStore` con Zustand y `persist` ; proteger las rutas de cliente y de administración con `ProtectedRoute` .

Calidad y entrega

TypeScript con `strict: true` y sin `any` ; nombres consistentes; `README.md` en la raíz con requisitos, instalación, ejecución, variables de entorno, usuario del seed y URL del repositorio. La organización del frontend por features se exige como arquitectura del proyecto.

Modelo de dominio

Entidades

Cinco tablas. Tipos: claves `int` autoincrementales, montos `float` y fechas `datetime` . Las restricciones (`≥ 0` , `≥ 1` , largos, valores permitidos) se validan en los schemas con `Field` y en el Service; no se exigen `CHECK` en la base.

Entidad	Campos	Reglas
Usuario	`id` , `username` (único), `full_name` , `email` (único), `hashed_password` , `role` (`user` o `admin`), `disabled` (default `false`), `created_at`	El registro público crea usuarios con rol `user` . Baja lógica con `disabled` .

Entidad	Campos	Reglas
Categoria	`id`, `nombre` (único, 2 a 100 caracteres), `descripcion`, `activo` (default `true`)	Baja lógica con `activo`.
Producto	`id`, `categoria_id` (FK a Categoria), `nombre`, `descripcion`, `precio_base` ($\geq$ `0`), `imagen_url`, `stock` ($\geq$ `0`, default 0), `disponible` (default `true`), `activo` (default `true`)	Pertenece a una categoría (1:N). Baja lógica con `activo`. Solo se vende si está activo, disponible y con stock suficiente.
Pedido	`id`, `usuario_id` (FK a Usuario), `estado` (default `PENDIENTE`), `forma_pago` (`EFECTIVO` o `TRANSFERENCIA`), `direccion_entrega` (5 a 200 caracteres), `notas` (opcional), `total`, `created_at`, `updated_at`	`total` = suma de los subtotales de sus líneas; fijo desde la creación. No se borra: se cancela.
DetallePedido	`id`, `pedido_id` (FK a Pedido), `producto_id` (FK a Producto), `cantidad` ($\geq$ `1`), `nombre_producto`, `precio_unitario`, `subtotal`	Línea inmutable con copia del nombre y del precio. `subtotal = precio_unitario × cantidad`.

Máquina de estados del pedido

Estado	Significado	Transiciones válidas	Quién
`PENDIENTE`	Pedido creado	a `EN_PREPARACION`, a `CANCELADO`	`admin` avanza o cancela; el cliente dueño cancela
`EN_PREPARACION`	Se está preparando	a `ENTREGADO`, a `CANCELADO`	`admin`
`ENTREGADO`	Terminal	ninguna	—
`CANCELADO`	Terminal	ninguna	—

El mapa de transiciones vive en el Service de pedidos como un diccionario; toda transición se valida antes de modificar el pedido.

Reglas de negocio

- **RN-01 Crear pedido.** Al menos un ítem y ningún producto repetido entre los ítems. Cada producto debe existir, estar activo y disponible y tener stock suficiente. Por cada ítem, en la misma iteración, se verifica el stock, se descuenta y se crea la línea copiando `nombre_producto` y `precio_unitario`; se calculan los subtotales y el `total`, y el pedido nace en `PENDIENTE`. Todo ocurre en una sola transacción: si falla una validación, no se persiste nada.
- **RN-02 Transiciones.** Solo las de la tabla de estados; un estado terminal no admite cambios. Una transición inválida responde 400.

- **RN-03 Cancelar.** Restituye el stock de cada línea en la misma transacción que cambia el estado.
- **RN-04 Permisos.** El cliente solo ve y cancela sus propios pedidos y solo desde `PENDIENTE`; el `admin` ve todos y puede avanzar o cancelar desde cualquier estado no terminal.
- **RN-05 Inmutabilidad.** Las líneas y el `total` no cambian después de crear el pedido. No hay endpoint de edición ni de borrado de pedidos.
- **RN-06 Datos del pedido.** `forma_pago` es `EFECTIVO` o `TRANSFERENCIA` y `direccion_entrega` tiene entre 5 y 200 caracteres. Se validan en el schema.

Roles y permisos

Rol	Permisos	Restricciones
`admin`	CRUD de categorías y productos, stock, ver todos los pedidos, avanzar y cancelar pedidos, resumen, listar, activar y desactivar usuarios. También puede crear pedidos como cualquier usuario con sesión.	No puede desactivarse a sí mismo.
`user`	Ver catálogo, crear pedidos, ver y cancelar sus pedidos (solo en `PENDIENTE`).	Solo accede a sus propios pedidos.

Especificación del backend

API REST construida con FastAPI y SQLModel sobre PostgreSQL. Expone todos los endpoints bajo `/api/v1` y documenta la API en `/docs` y `/redoc`.

Stack del backend

Las versiones se fijan en `requirements.txt`.

Tecnología	Versión	Rol
Python	3.11 a 3.13 (SQLModel 0.0.22 no funciona con 3.14)	Lenguaje
FastAPI	`fastapi[standard]` 0.115 o superior (el extra instala uvicorn y email-validator)	Framework REST y OpenAPI
SQLModel	0.0.22 o superior	ORM y schemas Pydantic v2
PostgreSQL + psycopg2-binary	PostgreSQL 15 o superior; psycopg2-binary 2.9	Base de datos. Para desarrollo local se acepta SQLite cambiando `DATABASE_URL`
python-jose[cryptography]	3.5.0	Firma y validación de JWT

Tecnología	Versión	Rol
Passlib + bcrypt	`passlib[bcrypt]==1.7.4` y `bcrypt==4.0.1` (fijar ambas: Passlib falla con bcrypt 5)	Hash de contraseñas
pydantic-settings, python-multipart	2.x; 0.0.20	Variables de entorno; formulario OAuth2 del login

Arquitectura en capas

Módulos por dominio y, dentro de cada módulo, la cadena Router, Service, Unit of Work, Repository y Model. El flujo de dependencias es unidireccional: ninguna capa importa de la capa superior.

Capa	Archivo	Responsabilidad	Conoce a
Router	`router.py`	HTTP: valida el cuerpo con schemas, resuelve sesión y rol con `Depends`, delega al Service y traduce las excepciones de dominio a `HTTPException`.	Service
Service	`service.py`	Reglas de negocio. Abre el Unit of Work con `with self.uow as uow:` y devuelve el schema de salida dentro del bloque.	Unit of Work
Unit of Work	`unit_of_work.py`	Crea la sesión, expone los repositorios que la comparten y hace commit o rollback en `__exit__`.	Repository, Session
Repository	`repository.py`	Consultas con `select`; hereda de `BaseRepository[T]`. Sin reglas de negocio.	Model, Session
Model	`models.py`	Tablas SQLModel (`table=True`) y relaciones.	Ninguna
Schemas	`schemas.py`	Modelos de entrada y salida (SQLModel sin `table=True`).	Ninguna

REGLA DEL UNIT OF WORK

El commit ocurre en el `__exit__` del Unit of Work. Ningún Service ni Router llama a `session.commit()`. El Router obtiene el Service con `Depends(get_<modulo>_service)` y nunca abre el Unit of Work. Esta regla se exige en `categorias`, `productos` y `pedidos`; los módulos `auth` y `usuarios` pueden conservar el estilo del proyecto de seguridad del que se parte. Se aceptan los nombres de archivo de ese proyecto (`model.py`, `core/uow.py`).

Componentes del núcleo (`app/core/`)

Componente	Archivo	Provee
Configuración	`config.py`	`Settings` con pydantic-settings: `DATABASE_URL`, `SECRET_KEY`, `ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `FRONTEND_ORIGIN`.
Base de datos	`database.py`	`engine`, `create_all_tables` y `session_factory`.
Seguridad	`security.py`	Hash y verificación con bcrypt, `create_access_token` y `decode_access_token`.
Dependencias	`deps.py`	`get_current_user` (lee la cookie), `get_current_active_user` (rechaza `disabled`) y `require_role`.
Unit of Work base	`unit_of_work.py`	Clase `UnitOfWork`: abre la sesión en `__enter__`; commit, rollback y close en `__exit__`.
Repositorio base	`base_repository.py`	`BaseRepository[T]` con `get_by_id`, `get_all`, `add` y `delete`.

Módulos de dominio (`app/modules/`)

Módulo	Contenido	Consultas propias del repositorio
`auth`	Registro, login con cookie HttpOnly, logout y perfil (`/me`). Usa el modelo Usuario.	—
`usuarios`	Modelo Usuario; listado, activación y desactivación por el `admin`.	`get_by_username`, `get_by_email`
`categorias`	CRUD con baja lógica y nombre único.	`get_by_nombre`
`productos`	CRUD, stock y disponibilidad, baja lógica, listado paginado con búsqueda.	`search` (nombre, categoría, disponible)
`pedidos`	Creación transaccional, máquina de estados, cancelación y resumen. Su Unit of Work expone los repositorios de pedidos y productos.	`list_by_usuario`, `count_by_estado`

Estructura de carpetas del backend

Cada módulo de `app/modules/` repite la misma cadena de archivos; se despliega `pedidos/` como ejemplo.

backend/	
app/	Paquete de la aplicación.
├── main.py	Crea la aplicación FastAPI, registra CORS con credenciales e incluye los routers con el prefijo /api/v1.
├── core/	Infraestructura compartida por todos los módulos.
│ ├── config.py	Settings con pydantic-settings, leídas de .env.
│ ├── database.py	engine, create_all_tables y session_factory.
│ ├── security.py	Hash bcrypt, create_access_token y decode_access_token.
│ ├── deps.py	get_current_user, get_current_active_user y require_role.
│ ├── unit_of_work.py	UnitOfWork base: commit, rollback y close en __exit__.
│ └── base_repository.py	BaseRepository[T]: get_by_id, get_all, add y delete.
├── modules/	Un paquete por dominio.
│ ├── auth/	router.py, service.py y schemas.py; usa el modelo Usuario.
│ ├── usuarios/	models, schemas, repository, unit_of_work, service y router.
│ ├── categorias/	Misma cadena de archivos que usuarios.
│ ├── productos/	Misma cadena de archivos que usuarios.
│ └── pedidos/	Dominio central, desplegado como ejemplo:
│ ├── models.py	Pedido y DetallePedido.
│ ├── schemas.py	PedidoCreate, PedidoRead, CambioEstado, ResumenPedidos y los demás schemas del módulo.
│ ├── repository.py	PedidoRepository: list_by_usuario y count_by_estado.
│ ├── unit_of_work.py	PedidoUnitOfWork: repositorios de pedidos y de productos sobre la misma sesión.
│ ├── service.py	RN-01 a RN-06, mapa de transiciones y excepciones de dominio.
│ └── router.py	get_pedido_service, endpoints de pedidos y traducción de errores.
└── db/	
└── seed.py	Carga admin, categorías y productos; idempotente.
requirements.txt	Dependencias con versiones fijadas.

Los nombres de archivo son orientativos; lo que se evalúa son las piezas y sus responsabilidades.

Modelos y relaciones

Las cinco entidades se implementan según el diagrama UML adjunto (Figura 1, en el anexo): nombres de campos, tipos, valores por defecto, claves y relaciones. Las relaciones se declaran con `Relationship` y `back_populates`, con estos nombres exactos, porque los schemas de lectura dependen de ellos:

Relación	Atributos
Categoria 1:N Producto	`Categoria.productos` y `Producto.categoria`
Usuario 1:N Pedido	`Usuario.pedidos` y `Pedido.usuario`
Pedido 1:N DetallePedido	`Pedido.items` y `DetallePedido.pedido`
Producto 1:N DetallePedido	`Producto.detalles` y `DetallePedido.producto`

Autenticación y autorización

Paso	Actor	Acción	Resultado
1	Cliente	`POST /api/v1/auth/token` con formulario OAuth2 (`username` , `password`)	200; el JWT viaja en la cookie HttpOnly `access_token` , que expira a los 30 minutos. El cuerpo no incluye el token.
2	Navegador	Adjunta la cookie en cada petición (Axios con `withCredentials: true`).	El frontend nunca lee ni guarda el token.
3	Backend	`get_current_user` lee la cookie, valida firma y expiración y carga el usuario; `get_current_active_user` rechaza usuarios con `disabled` .	401 sin sesión válida; 400 si la cuenta está desactivada.
4	Backend	`require_role(["admin"])` verifica el rol del usuario.	403 si el rol no está permitido.
5	Cliente	`POST /api/v1/auth/logout`	200; la respuesta borra la cookie.

API REST

Convenciones. Prefijo `/api/v1` ; documentación en `/docs` y `/redoc` . Los listados paginados reciben `skip` y `limit` (default 0 y 20, máximo 100) y devuelven `{ "total": N, "items": [...] }` ; opcionalmente pueden aceptar además `page` y `size` y devolver `page` , `size` y `pages` . Los listados públicos devuelven solo registros activos. Un registro dado de baja se trata como inexistente: su detalle, `PUT` , `PATCH` y `DELETE` responden 404. La baja lógica de una categoría no libera su nombre único ni oculta sus productos: para retirar una categoría del catálogo se dan de baja también sus productos. `PUT` actualiza los campos enviados y conserva los demás (actualización parcial). Activar y desactivar un usuario son operaciones idempotentes: responden 200 aunque el estado no cambie.

Errores. Todo error responde `HTTPException` con `detail` . Hay cuatro códigos comunes que no se repiten en las tablas:

Código	Cuándo	Quién lo produce
401	Falta la cookie o el token es inválido o venció, en cualquier endpoint con acceso «Sesión», «Dueño» o `admin`.	`get_current_user`
400 «Cuenta de usuario desactivada»	Usuario con `disabled = true`, en el login y en cualquier endpoint con sesión.	`get_current_active_user` y el login
403	El rol no está permitido, en cualquier endpoint con acceso `admin`.	`require_role`
422	El cuerpo o los parámetros no pasan la validación del schema.	FastAPI

Las tablas siguientes listan, en la columna «Errores», solo los códigos propios de cada endpoint. El Service los produce lanzando excepciones de dominio definidas en su `service.py`; el Router las captura con `try/except` y responde el código indicado.

Excepción de dominio	Código
`CategoriaNoEncontrada`, `ProductoNoEncontrado`, `PedidoNoEncontrado`, `UsuarioNoEncontrado`	404
`NombreDuplicado`, `UsuarioDuplicado`	409
`PedidoAjeno`	403
`StockInsuficiente`, `ProductoNoDisponible`, `ProductoRepetido`, `TransicionInvalida` y las demás reglas de negocio	400

En `auth` y `usuarios` se admite lanzar `HTTPException` directamente desde el Service, como en el proyecto de seguridad de la materia.

Auth y usuarios

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
POST	`/auth/register`	Registra un usuario con rol `user`.	Público	201 `UserPublic`	409 si `username` o `email` ya existen
POST	`/auth/token`	Login con formulario OAuth2; establece la cookie.	Público	200 `{ "message": "Login exitoso" }`	401 credenciales incorrectas; 400 cuenta desactivada

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
POST	`/auth/logout`	Borra la cookie.	Público	200	—
GET	`/auth/me`	Usuario autenticado.	Sesión	200 `UserPublic`	—
GET	`/admin/usuarios`	Listado de usuarios.	`admin`	200 `list[UserPublic]`	—
POST	`/admin/usuarios/{id}/activar`	Alta lógica (`disabled = false`).	`admin`	200 `UserPublic`	404
POST	`/admin/usuarios/{id}/desactivar`	Baja lógica (`disabled = true`).	`admin`	200 `UserPublic`	404; 400 si el `admin` intenta desactivarse a sí mismo

Categorías

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
GET	`/categorias`	Lista las activas.	Público	200 `list[CategoriaRead]`	—
GET	`/categorias/{id}`	Detalle.	Público	200 `CategoriaRead`	404
POST	`/categorias`	Crea; nombre único.	`admin`	201 `CategoriaRead`	409
PUT	`/categorias/{id}`	Actualiza.	`admin`	200 `CategoriaRead`	404, 409
DELETE	`/categorias/{id}`	Baja lógica (`activo = false`).	`admin`	204 sin cuerpo	404

Productos

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
GET	`/productos`	Lista activos. Query: `skip` , `limit` , `search` (sobre `nombre` , sin distinguir mayúsculas), `categoria_id` , `disponible` .	Público	200 `ProductosPaginados`	—

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
GET	`/productos/{id}`	Detalle con su categoría.	Público	200 `ProductoRead`	404
POST	`/productos`	Crea con `categoria_id`.	`admin`	201 `ProductoRead`	404 si la categoría no existe o está dada de baja
PUT	`/productos/{id}`	Actualiza los campos enviados.	`admin`	200 `ProductoRead`	404 producto inexistente; 404 si la nueva categoría no existe o está dada de baja
PATCH	`/productos/{id}/stock`	Actualiza `stock` y `disponible`.	`admin`	200 `ProductoRead`	404
DELETE	`/productos/{id}`	Baja lógica (`activo = false`).	`admin`	204 sin cuerpo	404

Pedidos

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
POST	`/pedidos`	Crea el pedido desde el carrito (RN-01).	Sesión	201 `PedidoRead`	404 producto inexistente o inactivo; 400 stock insuficiente, producto no disponible o repetido
GET	`/pedidos`	Propios (`user`) o todos (`admin`). Query: `skip`, `limit`, `estado`.	Sesión	200 `PedidosPaginados`	—
GET	`/pedidos/resumen`	Indicadores del panel. Se declara antes de `/{id}`.	`admin`	200 `ResumenPedidos`	—
GET	`/pedidos/{id}`	Detalle con sus líneas.	Dueño o `admin`	200 `PedidoRead`	404; 403 si el pedido es de otro usuario

Método	Endpoint	Descripción	Acceso	Respuesta	Errores
PATCH	`/pedidos/{id}/estado`	Avanza o cancela según la máquina de estados (RN-02, RN-03).	`admin`	200 `PedidoRead`	404; 400 transición inválida
PATCH	`/pedidos/{id}/cancelar`	Cancela un pedido propio en `PENDIENTE` (RN-03, RN-04).	Dueño	200 `PedidoRead`	404; 403 pedido ajeno; 400 si no está en `PENDIENTE`

Schemas

Los schemas son SQLModel sin `table=True`; los de lectura permiten `Modelo.model_validate(objeto)` dentro del Unit of Work. Nunca se expone un modelo de tabla como respuesta.

Schema	Campos	Validaciones
`UserCreate` / `UserPublic`	`username`, `full_name`, `email`, `password` (`UserPublic`: `id`, `username`, `full_name`, `email`, `role`, `disabled`, `created_at`; nunca `hashed_password`)	`password` mínimo 8 caracteres.
`CategoriaCreate` / `CategoriaUpdate` / `CategoriaRead`	`nombre`, `descripcion` (+ `id`, `activo` en Read)	`nombre` 2 a 100 caracteres.
`ProductoCreate` / `ProductoUpdate` / `ProductoRead`	`nombre`, `descripcion`, `precio_base`, `imagen_url`, `stock`, `disponible`, `categoria_id` (+ `id`, `activo`, `categoria: CategoriaRead` en Read)	`precio_base` ⩾ 0, `stock` ⩾ 0.
`ProductoStockUpdate`	`stock`, `disponible`	`stock` ⩾ 0.
`ProductosPaginados`	`total: int`, `items: list[ProductoRead]`	—
`ItemPedidoCreate`	`producto_id`, `cantidad`	`cantidad` ⩾ 1.
`PedidoCreate`	`items: list[ItemPedidoCreate]`, `forma_pago`, `direccion_entrega`, `notas`	Mínimo 1 ítem; `forma_pago` en (`EFECTIVO`, `TRANSFERENCIA`); `direccion_entrega` 5 a 200 caracteres.

Schema	Campos	Validaciones
`DetallePedidoRead`	`producto_id`, `nombre_producto`, `precio_unitario`, `cantidad`, `subtotal`	—
`PedidoRead`	`id`, `usuario_id`, `estado`, `forma_pago`, `direccion_entrega`, `notas`, `total`, `created_at`, `updated_at`, `items: list[DetallePedidoRead]`	—
`PedidosPaginados`	`total`, `items: list[PedidoRead]`	—
`CambioEstado`	`estado`	Uno de los cuatro estados.
`ResumenPedidos`	`pendientes`, `en_preparacion`, `entregados`, `cancelados`, `pedidos_hoy`, `ingresos_entregados`	Conteos con `func.count`; `ingresos_entregados` = suma de `total` de los pedidos `ENTREGADO`.

Flujo de crear pedido con Unit of Work

Paso	Capa	Operación	BD
1	Router	Recibe `POST /api/v1/pedidos`, valida `PedidoCreate` y obtiene el usuario con `get_current_active_user`.	No
2	Service	Abre `with self.uow as uow:`, valida que no haya productos repetidos y crea el `Pedido` en `PENDIENTE` con `total = 0`.	INSERT
3	Service	Por cada ítem: obtiene el producto con `uow.productos.get_by_id` (`ProductoNoEncontrado` si no existe o no está activo); verifica `disponible` (`ProductoNoDisponible`) y `stock ⩾ cantidad` (`StockInsuficiente`); descuenta `producto.stock`; crea el `DetallePedido` con `nombre_producto`, `precio_unitario` y `subtotal`; acumula el `total`.	Lectura, UPDATE, INSERT
4	Service	Asigna `pedido.total` con la suma de los subtotales.	UPDATE
5	Service	Convierte a `PedidoRead` con `model_validate` dentro del bloque y lo devuelve.	Lectura
6	UoW	`__exit__` sin excepción: commit y close. Con excepción: rollback y close; no se persiste nada, tampoco el descuento de stock.	COMMIT / ROLLBACK
7	Router	Responde 201 con el `PedidoRead`; `ProductoNoEncontrado` responde 404; `StockInsuficiente`, `ProductoRepetido` u otra regla, 400.	No

Configuración y ejecución del backend

Variable	Descripción	Valor de ejemplo
`DATABASE_URL`	Conexión a la base	`postgresql://user:pass@localhost:5432/foodstore_db` (desarrollo: `sqlite:///./foodstore.db`)
`SECRET_KEY`	Firma del JWT, mínimo 32 caracteres	`cambiar-esta-clave-de-al-menos-32-caracteres`
`ALGORITHM`	Algoritmo del JWT	`HS256`
`ACCESS_TOKEN_EXPIRE_MINUTES`	Expiración de la cookie y del token	`30`
`FRONTEND_ORIGIN`	Origen permitido en CORS	`http://localhost:5173`

`python -m app.db.seed` es idempotente: crea el usuario `admin` (`admin` / `Admin1234!`, rol `admin`), al menos 3 categorías y 8 productos con stock. Ejecución: `pip install -r requirements.txt`, copiar `.env.example` a `.env`, `python -m app.db.seed`, `uvicorn app.main:app --reload`.

Especificación del frontend

Aplicación React + TypeScript construida con Vite y estilada con Tailwind CSS. Consume la API con Axios, gestiona el estado del servidor con TanStack Query y el estado del cliente con Zustand.

Stack del frontend

Las versiones se fijan en `package.json`.

Tecnología	Versión	Rol
Node.js	20.19 o superior	Entorno de ejecución de las herramientas
React + TypeScript	19.x + 5.9 o superior	Interfaz
Vite	7 o superior	Build y servidor de desarrollo
Tailwind CSS	4.x con `@tailwindcss/vite`	Estilos
TanStack Query	5.x	Estado del servidor: fetching, caché e invalidación
TanStack Form	1.x	Formularios con validación

Tecnología	Versión	Rol
TanStack Table	8.x	Tablas del panel de administración
React Router	react-router-dom 7.x (`BrowserRouter`)	Navegación y rutas protegidas
Zustand	5.x	Estado global del cliente (sesión y carrito)
Axios	1.x	Cliente HTTP con `withCredentials` e interceptor de 401
pnpm	9 o superior	Gestor de paquetes

Arquitectura por features

El frontend se organiza por dominios: cada feature agrupa sus páginas, componentes, hooks, acceso a la API y tipos. Lo transversal vive en `shared/` , el estado global del cliente en `store/` y las rutas en `routes/` . Las features espejan los módulos del backend.

Pieza	Carpeta	Responsabilidad
Páginas	`pages/`	Componen la pantalla de una ruta: obtienen datos con hooks o directamente con `useQuery` y los pasan a los componentes.
Componentes	`components/`	Presentación con props tipadas; pueden leer los stores de sesión y carrito.
Hooks	`hooks/`	`useQuery` y `useMutation` de TanStack Query con `queryKey` descriptivas e invalidación.
API	`api/`	Funciones que llaman a los endpoints con la instancia de Axios y devuelven datos tipados.
Tipos	`types/`	Interfaces con los mismos nombres de campo que los schemas del backend.
Stores	`store/`	Estado del cliente con Zustand: sesión (`authStore`) y carrito (`cartStore`).

> **SEPARACIÓN DEL ESTADO**
>
> TanStack Query gestiona el estado del servidor: catálogo, pedidos, usuarios y resumen. Zustand gestiona el estado del cliente: sesión y carrito. Ningún componente obtiene datos del servidor con `fetch` dentro de `useEffect` ; la única llamada en `useEffect` es `checkSession` al montar la aplicación.

Componentes compartidos

Carpeta	Componentes	Uso
`shared/api/`	`axiosInstance.ts`, `handleApiError.ts`, `queryClient.ts`	Cliente HTTP único, mensajes de error a partir de `detail` e instancia de TanStack Query.
`shared/ui/`	`Button`, `Card`, `Input`, `Modal`, `BadgeEstado`	Piezas visuales base con Tailwind; `BadgeEstado` colorea los cuatro estados del pedido.
`shared/components/`	`Paginacion`, `Skeleton`, `EstadoVacio`, `EstadoError`	Estados de carga, vacío y error reutilizados en todas las listas.
`shared/layout/`	`Navbar`, `MainLayout`	`Navbar` con nombre y rol, «Cerrar sesión» y acceso al panel solo para `admin`; `MainLayout` reúne la `Navbar` y el contenido de la ruta.

Estructura de carpetas del frontend

Cada feature de `src/features/` repite la misma organización; se despliega `pedidos/` como ejemplo.

frontend/	
src/	
├── shared/	Transversal, sin lógica de negocio.
│ ├── api/	
│ │ ├── axiosInstance.ts	baseURL = VITE_API_URL, withCredentials e interceptor de 401.
│ │ ├── handleApiError.ts	Convierte el detail de la API en un mensaje para la pantalla.
│ │ └── queryClient.ts	Instancia única de TanStack Query.
│ ├── ui/	Button, Card, Input, Modal y BadgeEstado.
│ ├── components/	Paginacion, Skeleton, EstadoVacio y EstadoError.
│ └── layout/	Navbar y MainLayout.
├── features/	Un dominio por carpeta: pages, components, hooks, api y types.
│ ├── auth/	LoginPage, RegisterPage y UnauthorizedPage.
│ ├── categorias/	AdminCategoriasPage y useCategorias.
│ ├── productos/	CatalogoPage, AdminProductosPage, ProductoCard y useProductos.
│ ├── usuarios/	AdminUsuariosPage y useUsuarios.
│ └── pedidos/	Desplegado como ejemplo:
│ ├── pages/	CarritoPage, CheckoutPage, MisPedidosPage, AdminPedidosPage y AdminResumenPage.

│ ├── **components/**	Formulario de checkout, detalle del pedido, tabla de pedidos y tarjeta de resumen.
│ ├── **hooks/**	usePedidos, useCrearPedido, useCambiarEstadoPedido, useCancelarPedido y useResumenPedidos.
│ ├── **api/**	pedidos.ts: funciones que llaman a /pedidos.
│ └── **types/**	Pedido, DetallePedido, EstadoPedido y ResumenPedidos.
├── **store/**	
│ ├── authStore.ts	usuario, cargando, login, logout y checkSession; persist con partialize.
│ └── cartStore.ts	items, agregar, quitar, cambiarCantidad, vaciar y total; persist.
├── **routes/**	
│ ├── AppRouter.tsx	BrowserRouter y la tabla de rutas.
│ └── ProtectedRoute.tsx	Sin sesión redirige a /login; sin el rol, a /unauthorized.
├── App.tsx	Llama a checkSession al montar y renderiza AppRouter.
├── main.tsx	QueryClientProvider.
└── index.css	@import "tailwindcss".
.env.example	VITE_API_URL.
package.json	Dependencias con versiones fijadas.
vite.config.ts	Plugins react y @tailwindcss/vite.

Los nombres de archivo son orientativos; lo que se evalúa son las piezas y su comportamiento. Las consultas con TanStack Query pueden vivir en hooks propios o directamente en la página.

Rutas y pantallas

Ruta	Pantalla	Acceso	Qué hace
`/login`, `/register`	Formularios con TanStack Form	Público	Login con `username` y `password`; registro con validación.
`/`	Catálogo	Público	Lista paginada en el servidor (`skip = página × limit`) con búsqueda y filtro por categoría; skeleton mientras carga; estado vacío; botón «Agregar al carrito».
`/carrito`	Carrito	Público	Ítems, cantidades, subtotal y total; persiste al recargar.
`/checkout`	Confirmar pedido	Sesión	Formulario con `direccion_entrega` , `forma_pago` y `notas` ; envía `POST /pedidos` ; vacía el carrito y redirige a «Mis pedidos».

Ruta	Pantalla	Acceso	Qué hace
`/mis-pedidos`	Mis pedidos	Sesión	Lista y detalle de pedidos propios; botón cancelar solo en `PENDIENTE`.
`/admin/resumen`	Resumen	`admin`	Tarjetas con los indicadores de `ResumenPedidos`.
`/admin/categorias`, `/admin/productos`	ABM	`admin`	TanStack Table con paginación del lado del cliente (el panel pide `limit=100`); TanStack Form para crear y editar; cambio de stock y disponibilidad.
`/admin/pedidos`	Pedidos	`admin`	Tabla con filtro por estado y botones para avanzar o cancelar según la máquina de estados.
`/admin/usuarios`	Usuarios	`admin`	TanStack Table con `username`, nombre, email, rol y estado; botones Activar y Desactivar con `useMutation` e invalidación. El botón Desactivar no se muestra en la fila del usuario de la sesión.
`/unauthorized`	Sin permiso	Público	Mensaje y enlace al inicio.

Estado, datos y sesión

Pieza	Herramienta	Detalle
Estado del servidor	TanStack Query	`useQuery` para listados y detalles con `queryKey` descriptivas (`['productos', filtros]`); `useMutation` para crear, editar, borrar y cambiar estado, con `invalidateQueries` en `onSuccess`.
`authStore`	Zustand + `persist` con `partialize`	`usuario` (nombre, email y rol), `cargando` y las acciones `login`, `logout` y `checkSession` (llama a `/auth/me` al montar la aplicación). `partialize` guarda solo `usuario`; nunca el token.
`cartStore`	Zustand + `persist`	Ítems (`producto_id`, `nombre`, `precio_unitario`, `cantidad`), `agregar`, `quitar`, `cambiarCantidad`, `vaciar` y `total`.
Cliente HTTP	Axios	`baseURL = VITE_API_URL`, `withCredentials: true` e interceptor de respuesta: ante 401 fuera de `/auth/token` y `/auth/me`, o ante 400 con `detail` «Cuenta de usuario desactivada», limpia el `authStore`; la vuelta a `/login` la hace el propio interceptor o `ProtectedRoute` al quedar sin usuario.
Rutas protegidas	React Router	`ProtectedRoute` redirige a `/login` sin sesión y a `/unauthorized` si el rol no está permitido; muestra un indicador mientras `checkSession` confirma la sesión. La protección real está en el backend: ocultar un botón no reemplaza a `require_role`.
Logout	`authStore` + Axios	Llama a `POST /auth/logout`, limpia el `authStore` y redirige a `/login`.

Configuración y ejecución del frontend

Variable `VITE_API_URL` en `frontend/.env` (ejemplo: `http://localhost:8000/api/v1`). Ejecución: `pnpm install`, `pnpm build` (verifica los tipos) y `pnpm dev`.

Criterios de evaluación

Condición de aprobación. El trabajo se califica sobre 100 puntos y el resultado es **Aprobado** o **Desaprobado**. Se aprueba con 60 puntos o más, siempre que el proyecto corra siguiendo el `README.md` de la raíz; si no corre, se descuenta el 30 % de la nota antes de decidir el resultado. Un trabajo desaprobado recibe la devolución «Requiere correcciones» y se vuelve a presentar en la instancia siguiente con todas las observaciones resueltas y un video nuevo. Las instancias (primera entrega, recuperatorio en la primera mesa de examen y mesa de examen final) y sus fechas son las publicadas en el aula virtual.

El puntaje se asigna sobre el código entregado, según los criterios de la tabla.

#	Criterio	Pts	Puntaje completo	Qué resta
1	Backend: estructura y configuración	10	Módulos por dominio con las capas y los componentes del núcleo; `.env.example`; `requirements.txt` con versiones fijadas (incluye `bcrypt==4.0.1`); CORS con credenciales; seed idempotente.	Capas mezcladas, seed incompleto, dependencias sin fijar.
2	Modelo de datos	10	Las 5 entidades del UML con sus campos, relaciones con los nombres indicados, baja lógica y copia de nombre y precio en `DetallePedido`.	Entidades o campos faltantes, líneas sin copia de precio, pedidos borrables.
3	Unit of Work y Repository	10	`UnitOfWork` con `__exit__` (commit, rollback, close); `BaseRepository[T]`; en `categorias`, `productos` y `pedidos`, Services que abren el `with` y devuelven schemas dentro del bloque y Routers que no abren el Unit of Work.	`commit` en Service o Router; repositorios con reglas de negocio.
4	Autenticación y usuarios	10	Registro, login con cookie HttpOnly, logout y `/me` con los códigos 409, 401 y 400 indicados; `get_current_user`, `get_current_active_user` y `require_role`; escrituras de catálogo y gestión de pedidos y usuarios solo para `admin`; listado, activación y desactivación de usuarios (404, 400).	Token en el cuerpo o en localStorage; escrituras sin rol; códigos incorrectos; sin gestión de usuarios.

#	Criterio	Pts	Puntaje completo	Qué resta
5	API de catálogo	10	Endpoints de categorías y productos con verbos, códigos y schemas indicados; listado paginado con `search` y `categoria_id`; PATCH de stock; DELETE 204 lógico.	Códigos incorrectos, sin paginación, borrado físico.
6	API de pedidos y reglas de negocio	15	RN-01 a RN-06; errores con los códigos indicados; stock descontado y restituido en la misma transacción; resumen correcto.	Reglas faltantes, stock inconsistente, cambios de estado sin validar.
7	Frontend: cliente	10	Login y registro con TanStack Form; catálogo con búsqueda, filtro, paginación y skeleton; carrito persistido; checkout con TanStack Form; mis pedidos con detalle y cancelar.	Carencias funcionales, sin estados de carga o vacío.
8	Frontend: panel de administración	10	ABM de categorías y productos con TanStack Table y Form; stock; pedidos con cambio de estado; usuarios con activar y desactivar; resumen con tarjetas.	Falta alguna pantalla o solo visualización.
9	Frontend: estado, datos y sesión	10	TanStack Query para los datos del servidor, con invalidación; `authStore` y `cartStore` tipados con `persist`; Axios con `withCredentials` e interceptor; `ProtectedRoute` por rol.	`fetch` + `useEffect`; stores sin `persist`; rutas sin proteger.
10	Calidad y entrega	5	TypeScript `strict` sin `any`; nombres consistentes; `README.md` de la raíz con pasos que funcionan en una máquina limpia y la URL del repositorio.	`any` extendido, README incompleto.

Formato de entrega

Ítem	Descripción
CE-01	Archivo `.zip` del proyecto (sin `node_modules`, `venv` ni `.env`) y repositorio público de GitHub; la URL va en el `README.md` de la raíz.
CE-02	`README.md` en la raíz con requisitos, instalación y ejecución del backend y del frontend, variables de entorno y usuario `admin` del seed. Es el único README que se tiene en cuenta.
CE-03	`backend/.env.example` y `frontend/.env.example` completos.
CE-04	`python -m app.db.seed` carga `admin`, categorías y productos.

Ítem	Descripción
CE-05	`pip install -r requirements.txt` y `uvicorn app.main:app` sin errores; `/docs` muestra todos los endpoints.
CE-06	`pnpm install`, `pnpm build` y `pnpm dev` sin errores.
CE-07	Video de demostración de 10 a 15 minutos (link público en el README): presentación con la cámara encendida al inicio, registro, compra como cliente, gestión como `admin` y cancelación con restitución de stock.

Anexo: diagrama UML del modelo de datos

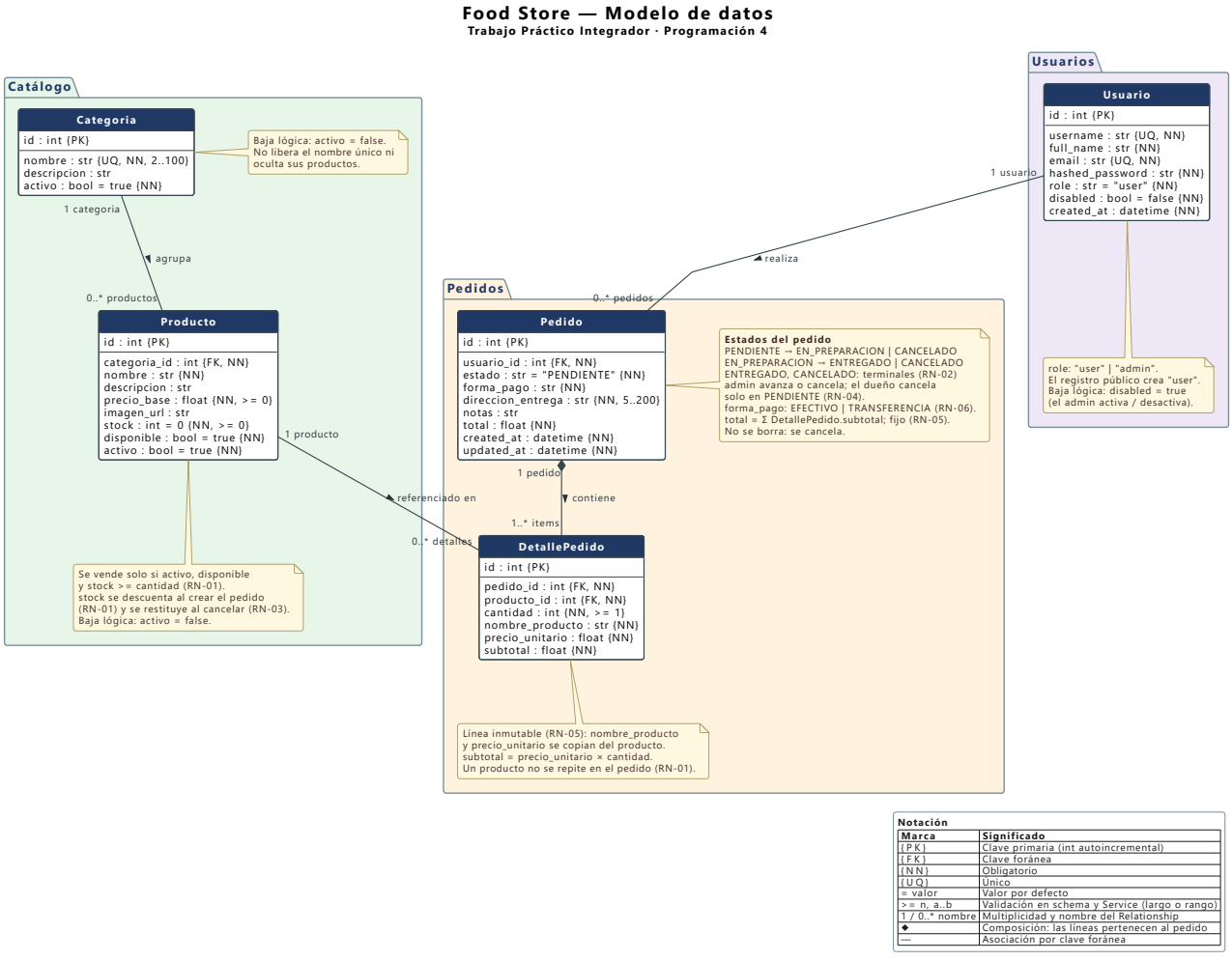


Figura 1. Diagrama UML del modelo de datos. Se adjunta como `UML_FOOD_STORE.svg` y `UML_FOOD_STORE.png`; el SVG permite ampliar sin pérdida. La leyenda del diagrama explica las marcas PK, FK, NN y UQ, los valores por defecto, las multiplicidades y las notas con las reglas de negocio.